TERM-II PROJECT REPORT

Password Strength Checker System

Object Oriented Programming (PCCCS495)

Prepared By: Souptik Khatua

Roll Number: 80

Department: Computer Science and Engineering

Instructor: Prof. Soumadip Biswas

College: Institute of Engineering & Management (IEM), Salt Lake

Submission Date: 29th March 2026

- **Spring** 2026

1. Problem Statement

In the modern era of cybersecurity, user credentials are the primary defense against unauthorized access. Many users tend to create simple, predictable passwords to avoid forgetting them, but these are highly vulnerable to dictionary and brute-force attacks. Current to-do applications or simple login systems often lack an intelligent feedback mechanism to guide users toward stronger security patterns. This project implements an object-oriented, rule-based Password Strength Checker that dynamically evaluates passwords based on standardized criteria—length, character complexity, and specific character sets—providing real-time suggestions to improve account security.

2. System Design

The system is architected using a modular approach to ensure extensibility and clean code. The core logic is decoupled from the user interface and data models through a serviceoriented structure.

2.1 Package Structure

- **PasswordStrengthChecker.main:** Controls the program flow and user input loop.
- **PasswordStrengthChecker.rules:** Contains the abstract Rule class and its concrete implementations.
- **PasswordStrengthChecker.service:** Handles the orchestration of rules and calculation of strength results.
- **PasswordStrengthChecker.model:** Defines the Result DTO (Data Transfer Object) for structured output.

2.2 Class Responsibilities

A central `StrengthChecker` class maintains a list of rules. When a password is entered, the service iterates through the collection, applying each rule's validation logic polymorphically.

3. OOP Concepts Demonstrated

3.1 Abstraction: The abstract `Rule` class defines the prototype for all security constraints. It encapsulates the shared attribute 'message' and mandates a common `validate()` behavior.

```
package PasswordStrengthChecker.rules;
public abstract class Rule {
protected String message;
    public Rule(String message) { this.message = message; }
public abstract boolean validate(String password);
public String getMessage() { return message; } }
```

3.2 Inheritance: Subclasses like `LengthRule` and `SpecialCharRule` inherit the `Rule` base class properties. This allows the system to share common traits while specializing logic.

3.3 Polymorphism: Method overriding is used in rule implementations. The `StrengthChecker` service treats all rules as generic types during execution, achieving runtime flexibility.

```
// In StrengthChecker.java - Dynamic Binding
for (Rule rule : rules) {
    if (rule.validate(password)) score++;
    else suggestions.add(rule.getMessage());
}
```

4. Testing & Results

The application was subjected to rigorous testing against various input scenarios, focusing on edge cases such as empty inputs and different levels of character complexity. Below are the execution logs captured during final testing.

Figure 1: Terminal output showing "Very Weak" classification for inputs with missing character types.

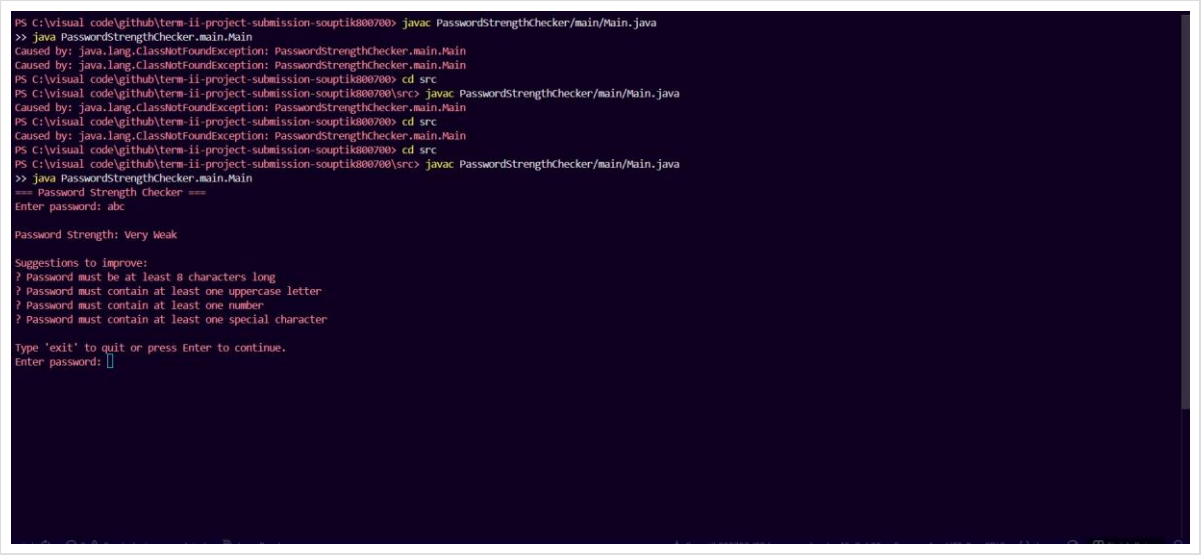


Figure 2: Terminal output showcasing a "Strong" password satisfying all architectural rules.

4.1 Robustness & Input Validation

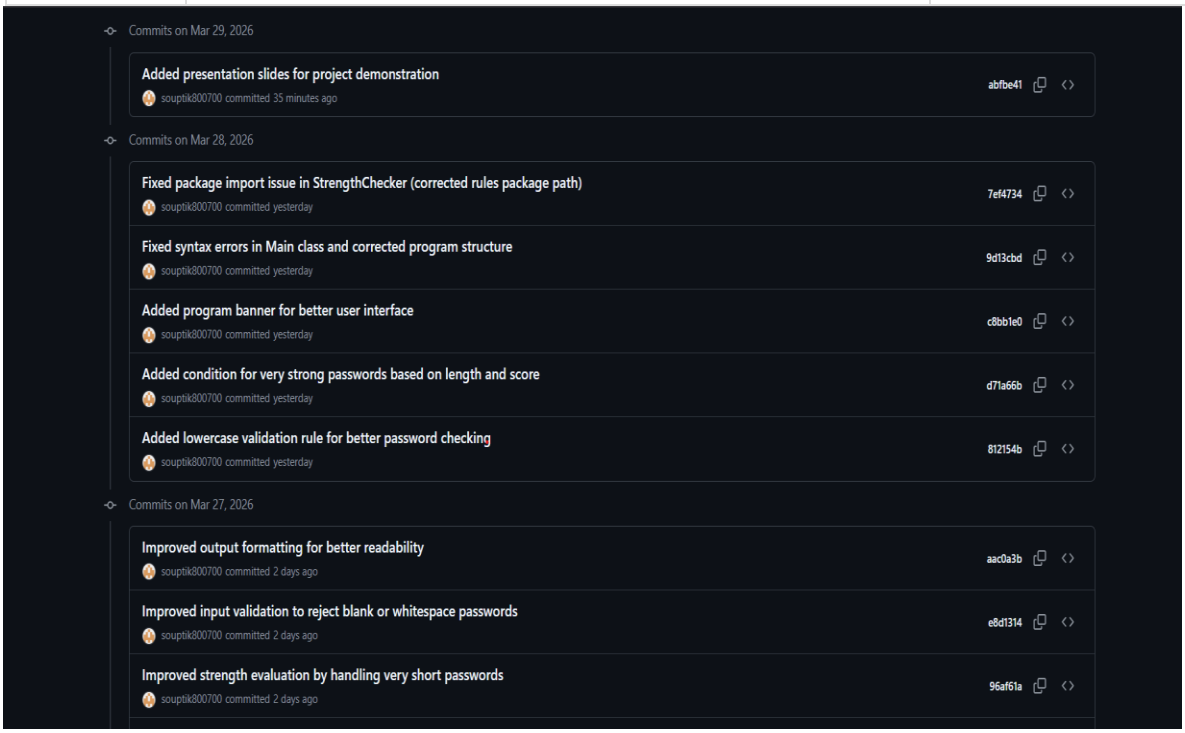
The system includes a dedicated input validation layer in the `Main` class to handle null or empty inputs. If a user provides an empty string, the system throws a descriptive error message instead of crashing, demonstrating effective exception handling strategies.

5. Git Workflow

Adhering to the course guidelines, the project was developed through 10+ granular commits spread across multiple development phases. The progression from the "Initial Setup" to "Logic Refinement" and "Final Documentation" is visible in the commit history.

Phase	Commit Description	Commit Type
1. Initial	Project structure and base abstract classes	Setup
2. Core	Implemented validation rules & strength logic	Feature

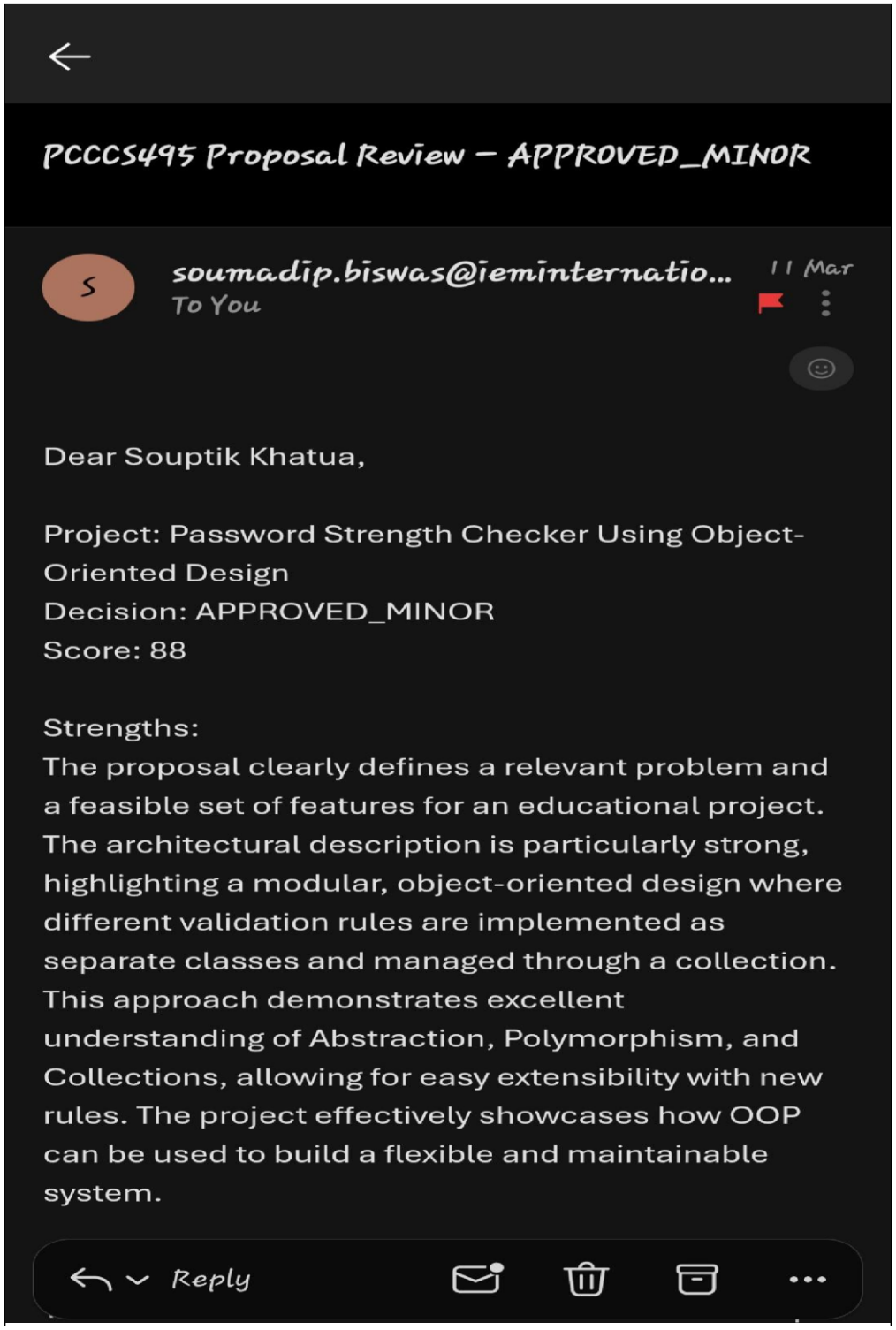
3. UI	Added Loop and better IO formatting	Enhancement
4. Fix	Corrected packages and fixed import errors	BugFix (Critical)



6. Appendix

6.1 Project Approval Record

The project proposal titled "Password Strength Checker Using Object-Oriented Design" was reviewed and approved with **MINOR_REVISIONS** by Prof. Soumadip Biswas. The following record serves as proof of approval for the final implementation phases.



PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@ieminternatio... 11 Mar
To You

Dear Souptik Khatua,

Project: Password Strength Checker Using Object-Oriented Design
Decision: APPROVED_MINOR
Score: 88

Strengths:
The proposal clearly defines a relevant problem and a feasible set of features for an educational project. The architectural description is particularly strong, highlighting a modular, object-oriented design where different validation rules are implemented as separate classes and managed through a collection. This approach demonstrates excellent understanding of Abstraction, Polymorphism, and Collections, allowing for easy extensibility with new rules. The project effectively showcases how OOP can be used to build a flexible and maintainable system.

Reply